OS & SP

Name : M Ramya Sree **Roll No :** 2520030094

SKILL – 11

1.Skill11.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_HISTORY 10

void trim(char *str)
{
    int start = 0;
    int end = (int)strlen(str) - 1;
```

```
    while (str[start] == ' ' || str[start] == '\t')
        start++;

    while (end >= start &&
           (str[end] == ' ' || str[end] == '\t'))
        end--;

    memmove(str, str + start, end - start + 1);
    str[end - start + 1] = '\0';
}

int main(void)
{
    char *history[MAX_HISTORY];
    int count = 0;
    char command[256];

    while (1) {
```

```
printf("ossp11$ ");
fflush(stdout);

if (fgets(command, sizeof(command), stdin) == NULL)
    break;

command[strcspn(command, "\n")] = '\0';

if (strcmp(command, "exit") == 0)
    break;

/* Store command */
if (count < MAX_HISTORY) {
    history[count] = strdup(command);

    if (history[count] == NULL) {
        perror("strdup");
        break;
```

```
    }

    count++;
}

/* Display history */
if (strcmp(command, "history") == 0) {
    for (int i = 0; i < count; i++)
        printf("%d %s\n", i + 1, history[i]);

    continue;
}

/* Find pipe */
char *pipe_symbol = strchr(command, '|');

if (pipe_symbol != NULL) {
    *pipe_symbol = '\0';
```

```
char *cmd1 = command;
char *cmd2 = pipe_symbol + 1;

trim(cmd1);
trim(cmd2);

int pipefd[2];

if (pipe(pipefd) == -1) {
   perror("pipe");
   continue;
}

/* First child */
pid_t p1 = fork();

if (p1 == -1) {
```

```
        perror("fork");
        close(pipefd[0]);
        close(pipefd[1]);
        continue;
    }
    if (p1 == 0) {
        close(pipefd[0]);
        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]);
        execlp(cmd1, cmd1, (char *)NULL);
        perror("exec");
        _exit(1);
    }
    /* Second child */
    pid_t p2 = fork();

    if (p2 == -1) {
        perror("fork");
```

```
        close(pipefd[0]);
        close(pipefd[1]);
        waitpid(p1, NULL, 0);
        continue;
    }
    if (p2 == 0) {
        close(pipefd[1]);
        dup2(pipefd[0], STDIN_FILENO);
        close(pipefd[0]);
        execlp(cmd2, cmd2, (char *)NULL);
        perror("exec");
        _exit(1);
    }

    close(pipefd[0]);
    close(pipefd[1]);
    waitpid(p1, NULL, 0);
    waitpid(p2, NULL, 0);
```

```
        } else {
            printf("Command stored: %s\n", command);
        }
    }
    for (int i = 0; i < count; i++)
        free(history[i]);
    return 0;
}
```

OUTPUT :

